

A Type Theory beyond Animae

Michele Riva

University of Manchester

j.w.w. Denis-Charles Cisinski and Tashi Walde

November 19, 2025

MOTIVATION

∞ -CATEGORY THEORY:

- Doing category theory “homotopically”.
- Get rid of strict equality of maps, replaced by homotopy.
- Language of algebraic topology.
- Foundations of mathematics alternative to set theory.

MOTIVATION

∞ -CATEGORY THEORY:

- Doing category theory “homotopically”.
- Get rid of strict equality of maps, replaced by homotopy.
- Language of algebraic topology.
- Foundations of mathematics alternative to set theory.

PROBLEMS:

- Lack of unified foundations.
- Set-theoretical models against the spirit of homotopy.
- Computationally-heavy proofs.

GOAL: Axiomatising ∞ -category theory within a type theory.

TYPE THEORY

TYPE THEORY: Derivation rules among **judgements**:

$\vdash A$ type

“ A is a type”

$\vdash a : A$

“ a is a term of type A ”

$\vdash A \equiv B$ type

“ A and B are equal types”

$\vdash a \equiv b : A$

“ a and b are equal terms”

TYPE THEORY

TYPE THEORY: Derivation rules among **judgements**:

$x : X \vdash A(x)$ type

“ A is a type...”

$x : X \vdash a(x) : A(x)$

“ a is a term of type A ...”

$x : X \vdash A(x) \equiv B(x)$ type

“ A and B are equal types...”

$x : X \vdash a(x) \equiv b(x) : A(x)$

“ a and b are equal terms...”

in **context** $x : X$.

TYPE THEORY

TYPE THEORY: Derivation rules among **judgements**:

$x : X, y : Y(x) \vdash A(x, y)$ type

“ A is a type...”

$x : X, y : Y(x) \vdash a(x, y) : A(x, y)$

“ a is a term of type A ...”

$x : X, y : Y(x) \vdash A(x, y) \equiv B(x, y)$ type

“ A and B are equal types...”

$x : X, y : Y(x) \vdash a(x, y) \equiv b(x, y) : A(x, y)$

“ a and b are equal terms...”

... in **context** $x : X, y : Y(x)$.

TYPE THEORY

TYPE THEORY: Derivation rules among **judgements**:

$x : X, y : Y(x) \vdash A(x, y)$ type "A is a type..."

$x : X, y : Y(x) \vdash a(x, y) : A(x, y)$ "a is a term of type A..."

$x : X, y : Y(x) \vdash A(x, y) \equiv B(x, y)$ type "A and B are equal types..."

$x : X, y : Y(x) \vdash a(x, y) \equiv b(x, y) : A(x, y)$ "a and b are equal terms..."

... in **context** $x : X, y : Y(x)$.

TYPES AS STATEMENTS:

- Statements about types and terms are formed as types.
- Terms of these statement types are the proofs.

TYPE THEORY

TYPE THEORY: Derivation rules among **judgements**:

$x : X, y : Y(x) \vdash A(x, y)$ type "A is a type..."

$x : X, y : Y(x) \vdash a(x, y) : A(x, y)$ "a is a term of type A..."

$x : X, y : Y(x) \vdash A(x, y) \equiv B(x, y)$ type "A and B are equal types..."

$x : X, y : Y(x) \vdash a(x, y) \equiv b(x, y) : A(x, y)$ "a and b are equal terms..."

... in **context** $x : X, y : Y(x)$.

TYPES AS STATEMENTS:

- Statements about types and terms are formed as types.
- Terms of these statement types are the proofs.

Identity type: $x : A, y : A \vdash \text{Id}_A(x, y)$ type is "x is equal to y".

Terms $\alpha : x =_A y$ give an internal notion of equality: **homotopy**

HOMOTOPY TYPE THEORY

HoTT: A type theory that describes types as ∞ -groupoids:

SYNTAX

- $\vdash B$ type
- $\vdash b : B$

INTUITION

- B ∞ -groupoid.
- $b : * \rightarrow B$.

HOMOTOPY TYPE THEORY

HOTT: A type theory that describes types as ∞ -groupoids:

SYNTAX

- $\vdash B$ type
- $\vdash b : B$
- $x : A \vdash B(x)$ type
- $x : A \vdash f(x) : B(x)$
- $x : A, y : A \vdash \text{Id}_A(x, y)$

INTUITION

- B ∞ -groupoid.
- $b : * \rightarrow B$.
- $B \twoheadrightarrow A$ isofibration.
- sections of $B \twoheadrightarrow A$.
- Path object of A .

HOMOTOPY TYPE THEORY

HOTT: A type theory that describes types as ∞ -groupoids:

SYNTAX

- $\vdash B$ type
- $\vdash b : B$
- $x : A \vdash B(x)$ type
- $x : A \vdash f(x) : B(x)$
- $x : A, y : A \vdash \text{Id}_A(x, y)$
- $\vdash A, B$ type $\rightsquigarrow x : A \vdash B$ type (weakening of B)
- $x : A \vdash f(x) : B$
- **Add pullback! And internal maps?**

INTUITION

- B ∞ -groupoid.
- $b : * \rightarrow B$.
- $B \twoheadrightarrow A$ isofibration.
- sections of $B \twoheadrightarrow A$.
- Path object of A .
- Given A, B ∞ -groupoids we have $B \times A \twoheadrightarrow A$.
- A section of $B \times A \twoheadrightarrow A$, that is a functor $A \rightarrow B$.

FUNCTORIALITY OF JUDGEMENTS

FUNCTORIALITY: Judgements are “functorial”:

$$x : A \vdash f(x) : B(x).$$

- This is mapping any $\vdash a : A$ into $a \vdash f(a) : B(a)$.

This gives a proof of a predicate about

FUNCTORIALITY OF JUDGEMENTS

FUNCTORIALITY: Judgements are “functorial”:

$$x : A \vdash f(x) : B(x).$$

- This is mapping any $\vdash a : A$ into $a \vdash f(a) : B(a)$.

This gives a proof of a predicate about all points in

FUNCTORIALITY OF JUDGEMENTS

FUNCTORIALITY: Judgements are “functorial”:

$$x : A \vdash f(x) : B(x).$$

- This is not just mapping any $\vdash a : A$ into $\vdash f(a) : B(a)$.
- For any $\alpha : S \rightarrow A$ we can assign $y : S \vdash f(\alpha(y)) : B(\alpha(y))$.

This gives a proof of a predicate about **all maps** into A .

FUNCTORIALITY OF JUDGEMENTS

FUNCTORIALITY: Judgements are “functorial”:

$$x : A \vdash f(x) : B(x).$$

- This is not just mapping any $\vdash a : A$ into $a \vdash f(a) : B(a)$.
- For any $\alpha : S \rightarrow A$ we can assign $y : S \vdash f(\alpha(y)) : B(\alpha(y))$.

This gives a proof of a predicate about **all maps** into A .

EXAMPLE: Let $\vdash A, C$ type and assume we can derive a rule

$$\frac{\vdash a : A}{\vdash f(a) : C}.$$

We **cannot** derive $x : A \vdash f(x) : C$.

Pointwise rules are **weaker** than judgements a priori.

FUNCTORIALITY OF JUDGEMENTS

FUNCTORIALITY: Judgements are “functorial”:

$$x : A \vdash f(x) : B(x).$$

- This is not just mapping any $\vdash a : A$ into $a \vdash f(a) : B(a)$.
- For any $\alpha : S \rightarrow A$ we can assign $y : S \vdash f(\alpha(y)) : B(\alpha(y))$.

This gives a proof of a predicate about **all maps** into A .

EXAMPLE: Let $\vdash A, C$ type and assume we can derive a rule

$$\forall \Gamma \frac{\Gamma \vdash a : A}{\Gamma \vdash f(a) : C}. \quad \text{We can derive} \quad x : A \vdash f(x) : C,$$

by applying the rule to the universal term: $\frac{x : A \vdash x : A}{x : A \vdash f(x) : C}.$

LIMITS OF HOMOTOPY TYPE THEORY

HoTT

The rules are the same in all contexts.

- Every pointwise assignment/property becomes functorial.
- If an embedding of types has a pointwise section, then it is an equivalence.
- Types are “statements”: they consist of points and homotopies.

MEANING

The theory of isofibrations of groupoids over a fixed Γ is the theory of groupoids.

“A property about all $* \rightarrow A$ becomes true for all sections of $A \times \Gamma \rightarrow \Gamma$, i.e. $\Gamma \rightarrow A$.”

LIMITS OF HOMOTOPY TYPE THEORY

HoTT

The rules are the same in **all** contexts.

- Every pointwise assignment/property becomes functorial.
- If an embedding of types has a pointwise section, then it is an equivalence.
- Types are “statements”: they consist of points and homotopies.

CATEGORY THEORY

The rules are the same only over **some** contexts.

- Assignments on objects between categories do not lift to functors.
- Want to access properties over the objects of a category only.
- Categories can be full “theories”, category theory describes itself.

GOALS

We want to construct a type theory where:

- (1) The rules are not the same in all contexts, to get
 - statements expressing objectwise, non-functorial properties.
 - Internal Homs (Π -types) only in some contexts.
- (2) We have directed, non-invertible, morphisms.
- (3) Universe types are not just “statements”: they describe the full theory of types and maps between them.

This will make the types richer, but harder to manipulate.

ANIMAE AND GROUPOID CORE

STRATEGY: We introduce a class of special contexts, “**animae**”:

$\vdash \Gamma \text{ anima.}$

- Formulate the theory so that the rules over any $\vdash \Gamma \text{ anima}$ are always the same as in empty context.
- Given $\vdash \Gamma \text{ anima}$, and $\Gamma \vdash A \text{ type}$, we say that A is a “**groupoid**” if $\vdash \Gamma, x : A \text{ anima}$.
- If A is a groupoid, we can manipulate A as in HoTT: everything about the points $\Gamma \vdash a : A$ will be lifted to context $\Gamma, x : A$ by structure of the theory.
 $\rightsquigarrow$ groupoids are the “**statements**” in the theory.

This distinction, i.e. the presence of non-groupoid types, is a new feature of the theory.

COMMON RULES WITH HoTT

We introduced an asymmetry in the theory, allowing us to formulate some rules only over in animae contexts. However, some rules of HoTT will still look the same, introduced in *arbitrary* contexts:

- Intensional identity types (we have a path object for isofibration of categories in every context). In particular we have a notion of equivalence between types.
- Σ -types (isofibrations of categories compose).
- Terminal type $\Gamma \vdash \Delta^0$ type (there is a terminal category/isofibration).

The usual rules apply to these types, and express the fact that we have a “tribe” structure on categories given by isofibrations. Let us now see the first differences.

TYPES OF MAPS

A map of types $\Gamma, x : A \vdash f(x) : B$ should be a functor $A \rightarrow B$ (over Γ).

- In HoTT, given $\Gamma, x : A \vdash B(x)$ type, we can form the Π -type $\Gamma \vdash \Pi_{x:A} B(x)$ giving access to:
 - internal Homs $\Gamma \vdash X \rightarrow Y$ type whose terms are maps.
 - $\forall$ connective to form statements.

This is the existence of a right adjoint to the pullback along an isofibration of groupoids.

- In this theory, we can form types of maps over animae, but not in general: not all isofibrations are exponentiable. Given $\vdash \Gamma$ anima and $\Gamma \vdash X, Y$ type we can either:
 - (1) introduce functor categories $\text{Fun}(X, Y)$ as right adjoints. These are not groupoids.
 - (2) Introduce a groupoid of maps $\text{Map}(X, Y)$.

THE IMPORTANCE OF YONEDA

MAPPING SPACES

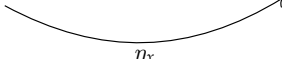
GROUPOID CORE

Π -TYPES

OBJECTWISE STATEMENTS

We get access to objectwise statements: assume we defined an interval $\vdash \Delta^1$ type, hence (iso)morphisms in a type.

Ex: Want to characterise $\eta : \Delta^1 \times X \rightarrow Y$ being a natural iso:

$$\Delta^1 \times * \xrightarrow{1_{\Delta^1} \times x} \Delta^1 \times X \xrightarrow{\eta} Y \quad \text{iso for all } x.$$


OBJECTWISE STATEMENTS

We get access to objectwise statements: assume we defined an interval $\vdash \Delta^1$ type, hence (iso)morphisms in a type.

Ex: Want to characterise $\eta : \Delta^1 \times X \rightarrow Y$ being a natural iso.

IN MLTT: we could only write

$x : X \vdash \sigma(x) : \{\eta_x \text{ is an isomorphism}\},$ which means

$$\Delta^1 \times S \xrightarrow{1_{\Delta^1} \times x} \Delta^1 \times X \xrightarrow{\eta} Y \quad \text{iso for all } \vdash S \text{ type and } x.$$

Not the weaker condition we want! Could take $S = X$.

OBJECTWISE STATEMENTS

We get access to objectwise statements: assume we defined an interval $\vdash \Delta^1$ type, hence (iso)morphisms in a type.

Ex: Want to characterise $\eta : \Delta^1 \times X \rightarrow Y$ being a natural iso

IN OUR SETTING: The objectwise condition can be written as

$x : X^\simeq \vdash \sigma(x) : \{\eta_x \text{ is an isomorphism}\},$ which means

$$\Delta^1 \times \Gamma \xrightarrow{1_{\Delta^1} \times x} \Delta^1 \times X \xrightarrow{\eta} Y \quad \text{iso for all } \vdash \Gamma \text{ gpd and } x.$$

We can finally formulate objectwise conditions in this settings.